

Validation Logic for Link Verification Using Playwright and Excel Data

Objective

The objective of this validation is to verify that all links maintained in an Excel file are available on the web application and functioning correctly. The automation solution should compare the expected links from the Excel file with the links displayed on the web page and identify any missing or broken links. The final result should provide a clear PASS or FAIL status for each link based on the validation outcome.

Validation Logic

1. Read all expected links from the Excel file.
2. Launch the application and navigate to the target page.
3. Retrieve all links displayed on the web page.
4. Compare the links from the Excel file with the links available on the page.
5. Verify whether each expected link is present on the web page.
6. If a link is not found, mark it as **FAIL**.
7. If the link is found, retrieve its URL (**href** attribute).
8. Validate that the URL is not empty or null.
9. Check the URL response status to ensure the link is accessible.
10. Mark the link as:
 - **PASS** if the link is present and returns a successful response.
 - **FAIL** if the link is missing or returns an error response.
11. Generate a report containing the link name, URL, and validation status for all links.

Expected Result

- All links in the Excel file should be available on the web page.
- Each link should open successfully.
- Missing or broken links should be marked as FAIL.
- A report should show the status of each link.

Sample Implementation

The validation can be automated using Playwright by reading the expected links from an Excel file and comparing them with the links displayed on the web page. The following TypeScript code shows a simple example of how the validation can be performed.

```
import { test, expect } from '@playwright/test';

test('Validate links from web page', async ({ page }) => {

    await page.goto('https://example.com');

    const expectedLinks = ['Home', 'About Us', 'Contact'];

    for (const linkText of expectedLinks) {

        const link = page.getByRole('link', { name: linkText });

        if (await link.isVisible()) {
            console.log(`${linkText} - PASS`);
        } else {
            console.log(`${linkText} - FAIL`);
        }
    }
});
```

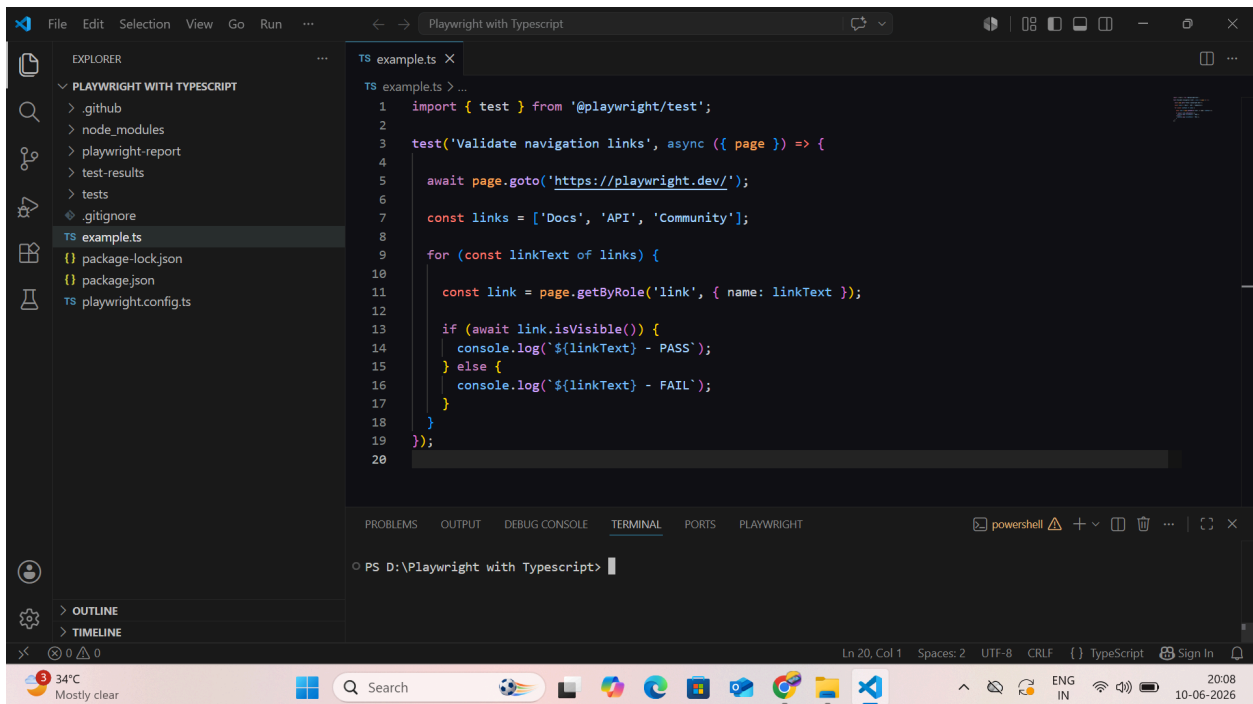
Code Explanation

The script first opens the web page and defines a list of expected links. It then loops through each link and checks whether the link is visible on the page using Playwright. If the link is found, the script marks it as PASS; otherwise, it marks it as FAIL. This is a

simple example that demonstrates the basic validation logic for checking the presence of links on a web page.

Conclusion

The above approach can be used to validate links available on a web page by comparing them with the expected links from an Excel file. It helps identify missing links and ensures that the required links are displayed correctly. The same approach can be extended to validate the link URLs and check for broken links if needed.



```
TS example.ts X
TS example.ts > ...
1  import { test } from '@playwright/test';
2
3  test('Validate navigation links', async ({ page }) => {
4
5      await page.goto('https://playwright.dev/');
6
7      const links = ['Docs', 'API', 'Community'];
8
9      for (const linkText of links) {
10
11          const link = page.getByRole('link', { name: linkText });
12
13          if (await link.isVisible()) {
14              console.log(`${linkText} - PASS`);
15          } else {
16              console.log(`${linkText} - FAIL`);
17          }
18      }
19  });
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT

PS D:\Playwright with Typescript>

Sample Validation Logic

Playwright Test Report

localhost:9323

Search tests

All 6Passed 6Failed 0Flaky 0Skipped 0

10/6/2026, 8:12:12 pmTotal time: 10.4s

example.spec.ts

✓ has titlechromium5.1s

example.spec.ts:3

✓ get started linkchromium5.5s

example.spec.ts:10

✓ has titlefirefox4.3s

example.spec.ts:3

✓ get started linkfirefox3.8s

example.spec.ts:10

✓ has titlewebkit2.6s

example.spec.ts:3

✓ get started linkwebkit3.9s

example.spec.ts:10

34°C
Mostly clear

Search

ENG
IN

20:12
10-06-2026

Validation Results